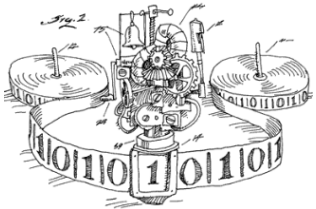
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Sesión 14. Control Supervisor Descentralizado (SCT): Síntesis de Supervisores y Prevención de Deadlocks

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 14

UNIVERSIDAD DE LOS ANDES VENEZUELA

Fundamentos de la Teoría de Control Supervisor (SCT)

Condición de Controlabilidad y No-Bloqueo

Síntesis de Supervisores en Redes de Petri

Arquitectura de Supervisión Distribuida y Código

Síntesis General del Curso (Semanas 1 a 14)

UNIVERSIDAD DE LOS ANDES VENEZUELA

De la Verificación Pasiva al Control Activo: La Teoría de Ramadge-Wonham

¿Qué hacemos cuando el proceso abierto puede bloquearse?

UNIVERSIDAD DE LOS ANDES VENEZUELA

El Límite de la Verificación en Lazo Abierto

En la Sesión 12-A aprendimos a diagnosticar si una red contiene estados inseguros o riesgo de *deadlock* (sifones que se vacían). Pero en la industria real:

- No podemos cambiar la física de las máquinas ni eliminar la compartición de recursos costosos (robots, reactores, bombas de trasvase).
- La planta libre G genera por naturaleza secuencias legítimas mezcladas con trayectorias que conducen a colisiones o bloqueos mutuos.

La Solución del Control Supervisor (Ramadge & Wonham, 1987)

En lugar de rediseñar los recursos locales, acoplamos un agente lógico llamado **Supervisor (S)** que observa los eventos en tiempo real y **restringe dinámicamente qué acciones pueden iniciarse** para mantener al sistema dentro de una región segura y libre de bloqueos.

La Partición Fundamental del Alfabeto: $\Sigma = \Sigma_c \dot{\cup} \Sigma_{uc}$

UNIVERSIDAD DE LOS ANDES VENEZUELA

No todos los eventos obedecen órdenes

Para ejercer control real sobre un Sistema de Eventos Discretos (DES), el alfabeto Σ se divide estrictamente en dos conjuntos disjuntos:

1. Eventos Controlables (Σ_c)

Transiciones que el supervisor puede **habilitar o deshabilitar** a voluntad mediante comandos de software o señales de PLC:

- `iniciar_carga`, `asignar_robot`
- `abrir_valvula`, `aprobar_orden`

Son las ≪válvulas de admisión≫ del flujo de trabajo.

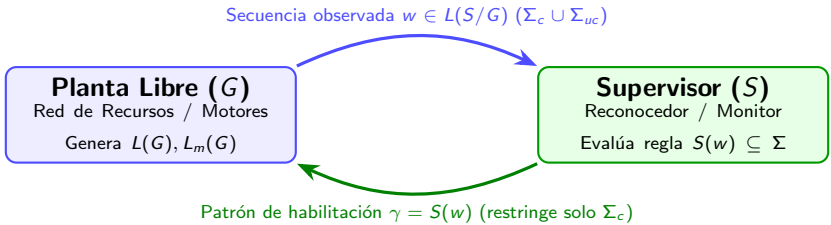
2. Eventos Incontrolables (Σ_{uc})

Eventos espontáneos de la física o del entorno que el supervisor **solo puede observar, jamás impedir**:

- `fin_mecanizado`, `pieza_lista`
- `falla_motor`, `cancelacion_cliente`

Prohibir por software una avería física es imposible.

El Lazo de Control Supervisor S/G



Definición Formal de la Ley de Control

Un supervisor es un mapeo $S : L(G) \to 2^{\Sigma}$ que, tras observar la traza histórica w, define el conjunto de eventos permitidos $S(w)$ cumpliendo la **condición de admisibilidad**:

$$\forall w \in L(G): \quad \Sigma_{uc} \subseteq S(w)$$

El evento $e \in \Sigma$ puede ocurrir en el sistema supervisado S/G si y solo si está habilitado en la planta ($we \in L(G)$) **y** autorizado por el supervisor ($e \in S(w)$).

Teorema de Controlabilidad y No-Bloqueo: ¿Cuándo existe un Supervisor Válido?

El Teorema de Controlabilidad de Ramadge-Wonham

UNIVERSIDAD DE LOS ANDES VENEZUELA

¿Cuándo podemos imponer una especificación segura $K \subseteq L_m(G)$?

Sea K el lenguaje de todas las trazas deseables y seguras. Existe un supervisor admisible S tal que $L(S/G) = \overline{K}$ si y solo si K es **controlable** respecto a G y Σ_{uc}:

$$\overline{K}\,\Sigma_{uc} \cap L(G) \subseteq \overline{K}$$

Interpretación Intuitiva en Planta

Si el proceso se encuentra en un estado seguro dentro de $\overline{K}$ y ocurre de forma espontánea cualquier evento incontrolable $\sigma_u \in \Sigma_{uc}$ físicamente posible en la planta, **el nuevo estado alcanzado no puede salirse de la zona segura** $\overline{K}$.

Consecuencia de Diseño

Si un evento incontrolable lleva al desastre, el supervisor debe **cortar el camino antes**, deshabilitando el último evento controlable que dio acceso a ese estado de riesgo.

Supervisión No-Bloqueante (*Non-Blocking* SCT)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Evitar el ≪Supervisor Paranoico≫

Un supervisor que deshabilita absolutamente todos los eventos controlables ($S(w) = \Sigma_{uc}$) jamás entra en un estado prohibido, pero paraliza la fábrica. Exigimos que el sistema supervisado sea **No-Bloqueante**:

$$\overline{L_m(S/G)} = L(S/G)$$

Significado en Workflows

- Toda traza parcial autorizada por el supervisor $w \in L(S/G)$ es prefijo de al menos una traza completa que llega al estado marcado final ($wv \in L_m(S/G)$).
- Ninguna orden de producción autorizada a entrar al sistema quedará atrapada en un callejón sin salida.

El Supremo Controlable $\sup \mathcal{C}(K)$

Cuando K no es controlable, el algoritmo de síntesis poda iterativamente los estados malos y de bloqueo hasta obtener el **sublenguaje controlable máximo** $\sup \mathcal{C}(K)$: la supervisión *mínimamente restrictiva*.

UNIVERSIDAD DE LOS ANDES VENEZUELA

Supervisión en Redes de Petri: Control por Invariantes y Lugares Monitores

De Autómatas a Redes de Petri: Evitando la Explosión de Estados

El Reto Computacional del Producto Síncrono

Construir el autómata global $G = G_1 \parallel G_2 \parallel \cdots \parallel G_n$ para calcular $\sup \mathcal{C}(K)$ sufre una explosión exponencial de estados. En la Sesión 13 vimos que nuestro motor industrial se basa en **Redes de Petri matriciales**.

Síntesis Algebraica mediante Invariantes de Lugar (Moody & Antsaklis)

En una Red de Petri con matriz de incidencia C_p, las especificaciones de seguridad y prevención de bloqueos se expresan como **Restricciones Lineales de Exclusión Mutua Generalizada (GMEC)**:

$$L \cdot M_p \leq b$$

donde $M_p \in \mathbb{N}^{|P|}$ es el marcado de la planta, L una matriz de pesos enteros y b el vector de cotas máximas permitidas.

Construcción Automática de Lugares Monitores (P_c)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Transformación de Desigualdad a Invariante con Variable de Holgura

Para forzar $L \cdot M_p \leq b$ en todo instante, introducimos un conjunto de **Lugares Monitores (Supervisores)** P_c cuyo marcado $M_c \geq 0$ actúa como variable de holgura (*slack*):

$$L \cdot M_p + M_c = b$$

1. Matriz de Incidencia del Supervisor (C_c)

Como el invariante debe cumplirse para cualquier disparo ($L \cdot \Delta M_p + \Delta M_c = 0$):

$$C_c = -L \cdot C_p$$

2. Marcado Inicial del Monitor (M_{c0})

En el arranque del motor ($k = 0$):

$$M_{c0} = b - L \cdot M_{p0}$$

Resultado brillante: ¡El supervisor es simplemente un conjunto de nuevas filas C_c añadidas a la matriz de incidencia C del mismo motor de Redes de Petri de la Sesión 13!

Caso Industrial: Prevención de Deadlock en Recursos Compartidos

El Problema: Disputa Circular (R_1, R_2)

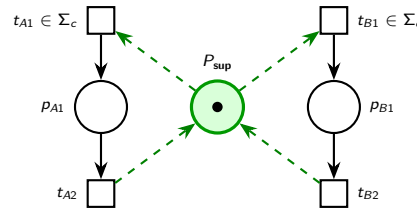
- **Línea A:** Usa primero R_1 (p_{A1}) y luego necesita R_2 (p_{A2}) antes de liberar R_1.
- **Línea B:** Usa primero R_2 (p_{B1}) y luego necesita R_1 (p_{B2}) antes de liberar R_2.
- **Deadlock:** Si $M(p_{A1}) = 1$ y $M(p_{B1}) = 1$ al mismo tiempo, ambas líneas retienen un recurso y esperan eternamente por el otro.

Síntesis del Lugar Monitor (P_{sup})

Imponemos la restricción GMEC sobre los estados de entrada:

$$M(p_{A1}) + M(p_{B1}) \leq 1$$

Se añade el lugar monitor P_{sup} con $M_0(P_{\text{sup}}) = 1$, que consume ficha en $t_{A1}, t_{B1} \in \Sigma_c$ y la recupera en t_{A2}, t_{B2}.



El lugar monitor P_{sup} (verde) evita que el sifón de recursos se vacíe.

UNIVERSIDAD DE LOS ANDES VENEZUELA

Arquitectura de Supervisión Distribuida: Implementación en el Motor Fénix

Del Supervisor Centralizado al Control Modular / Holónico

UNIVERSIDAD DE LOS ANDES VENEZUELA

¿Cómo supervisar una red distribuida de Holones de Recurso?

En el enfoque holónico (Fénix), no existe un único supertallador central que gobierne cada válvula interna de cada máquina:

1. Autonomía Local (Intra-Holón)

- Cada Holón Recurso gestiona sus sub-procesos internos (PDA / RdP local).
- Las transiciones puramente internas son invisibles para el resto de la planta.

2. Supervisores Modulares (Inter-Holón)

- Por cada zona de conflicto o trasvase compartido se instancia un **Monitor Modular** S_k.
- Un evento controlable compartido $e \in \Sigma_c$ se autoriza por conjunción distribuida:

$$e \in \bigcap_{k \in \mathcal{M}(e)} S_k(M)$$

Especificación YAML con Controlabilidad y Monitores SCT

Transiciones Tipadas (Σ_c vs. Σ_{uc})

```
transiciones:
  - id: "t_A1"
    trigger: "INICIAR_LOTE_A"
    controlable: true    # Sigma_c
    pre:  { p_espera_A: 1, R1_libre: 1 }
    post: { p_A1: 1 }

  - id: "t_falla_R1"
    trigger: "ALARMA_TERMICA"
    controlable: false
# Sigma_uc (Incontrolable)
    pre:  { p_A1: 1 }
    post: { p_mant_R1: 1 }
```

Restricción GMEC del Supervisor

```
supervisores_gmec:
  - id: "P_sup_deadlock_R1_R2"
    descripcion: "Evita bloqueo circular A-B"
    pesos_L:
      p_A1: 1
      p_B1: 1
    cota_b: 1
```

El cargador lee supervisores_gmec y sintetiza automáticamente la fila $C_c = -L \cdot C_p$ y el marcado inicial $M_{c0} = b - L \cdot M_{p0}$.

Implementación en Python: Síntesis e Integración SCT

UNIVERSIDAD DE LOS ANDES VENEZUELA

Inyección de Lugares Monitores y Verificación de Admisibilidad

```
class SupervisedPetriNet(PetriNet):
    def add_gmec_monitor(self, monitor_id, L_weights, b_bound):
        # 1. Calcular fila de incidencia del monitor: C_c = - L * C_p
        c_row = [0] * self.num_t
        for t in range(self.num_t):
            c_row[t] = -sum(L_weights.get(self.p_names[p], 0) * self.C[p][t]
                            for p in range(self.num_p))
            # 2. Verificar admisibilidad de Ramadge-Wonham:
            # Un monitor NUNCA puede consumir fichas en una transicion incontrolable
            if c_row[t] < 0 and not self.is_controllable[t]:
                raise ValueError(f"GMEC no admisible: intenta frenar t={t} incontrolable")

        # 3. Calcular marcado inicial de holgura: M_c0 = b - L * M_p0
        m0_c = b_bound - sum(L_weights.get(self.p_names[p], 0) * self.M[p]
                             for p in range(self.num_p))
        self.append_place(monitor_id, m0_c, c_row)
```

UNIVERSIDAD DE LOS ANDES VENEZUELA

Cierre del Curso: El Mapa Completo de la Automatización Formal

De la Jerarquía de Chomsky a la Planta Holónica Supervisada

UNIVERSIDAD DE LOS ANDES VENEZUELA

Integración de todos los Formalismos del Semestre

Bloque / Semanas	Formalismo Teórico	Artefacto de Software	Rol en la Automatización (Fénix)
1. Fundamentos (S1–S3)	Lógica Combinatoria y BNF	Gramáticas EBNF / Reglas	Definir vocabulario y sintaxis de procesos
2. Estado Local (S4–S7)	DFA (Tipo 3) y RegEx	`recurso.py` (Hito 1)	Secuencias lineales y ciclos de un recurso
3. Recursividad (S8–S10)	PDA (Tipo 2) y Lema Bombeo	`recurso_pda.py` (Hito 2)	Parser YAML y sub-workflows holónicos (LIFO)
4. Concurrencia (S11–S13)	Redes de Petri y Álgebra C	`recurso_petri.py` (Hito 3)	Paralelismo, recursos compartidos, Watchdogs
5. Control SCT (S14)	Ramadge-Wonham y GMEC	`supervisor_sct.py` (Final)	Prevención activa de deadlocks (Σ_c vs. Σ_{uc})

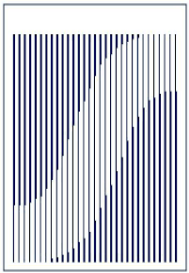
Mensaje Final del Curso

La **Teoría de la Computación** no es un ejercicio abstracto de pizarrón: es la herramienta de ingeniería que nos permite **especificar, parsear, verificar matemáticamente, simular y supervisar en tiempo real** sistemas distribuidos complejos con garantía formal de seguridad y ausencia de bloqueos.

¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve